

Report

Comparison of von Heijne and Support Vector Machine Models for Signal Peptide Prediction

Bianca Gustini^{1,*}, Amir Aynede¹, Kianoush Keshani¹ and Priscilla Castro-Vargas¹

¹Department of Pharmacy and Biotechnology, Alma Mater Studiorum – Università di Bologna, Address Via F. Selmi 3 - 4th Floor, Room 183.

*To whom correspondence should be addressed.

Associate Editor: Castrense Savojardo

Received on 20/11/2025

Abstract

Motivation: Signal peptides direct proteins into secretory pathway in Eukaryotes. Given that their experimental identification is costly, time-consuming and labor-intensive, the development of *in silico* methods to predict signal peptide-presence in proteins is of critical importance. Indeed, signal peptides play major roles in several biological processes of substantial interest in numerous research fields (e.g. genome annotation, disease diagnosis). This work compares the performances of von Heijne and SVM models in the classification of protein sequences as signal peptide-carrying or not.

Results: As expected, the statistical nature of the von Heijne method led it to perform well on canonical motifs and to misclassify sequences carrying a signal peptide characterized by a non-canonical composition. The SVM, trained on context-based physicochemical features reached higher performance overall, resulting to be far more accurate in the signal peptide prediction.

Availability: All code, preprocessing scripts, trained models, and benchmarking datasets used in this study are openly available in the project's GitHub repository: <https://github.com/AmirAynede/LB2-project-Group-1>. The repository provides the complete workflow for dataset assembly, feature extraction, von Heijne scoring, SVM training, cross-validation, and benchmarking, enabling full reproducibility of the analyses.

Contacts: bianca.gustini@studio.unibo.it, amir.aynede@studio.unibo.it, kianoush.keshani@studio.unibo.it, priscilla.castro@studio.unibo.it.

Supplementary information: The online version contains supplementary material available at <https://github.com/AmirAynede/LB2-project-Group-1>.

1 Introduction

The intracellular distribution of proteins is tightly regulated, as each protein must be delivered to the subcellular compartment that provides the appropriate biochemical environment for its function [1]. Cells rely on several protein-sorting mechanisms, many of which depend on N-terminal targeting signals [1]. Disruption of these pathways, or protein mislocalization, is associated with a wide range of human diseases [2].

Signal peptides (SPs) are N-terminal sequences of 15–30 residues that direct proteins to the secretory pathway in eukaryotes and mediate translocation across the plasma or inner membrane in prokaryotes [3,4]. As shown in **Figure 1**, SPs consist of three regions: (i) N-region, enriched in positively charged residues such as lysine, arginine, and histidine; (ii) H-region, the hydrophobic core of the SP, which often adopts an α -helix conformation; and (iii) C-region, containing neutral or polar residues and a weakly conserved cleavage site, that allows for precise cleavage after translocation [5]. SPs play key roles across diverse fields,

including genome annotation, recombinant protein production, disease diagnostics and vaccine design [3–5].

Given that SPs direct proteins into the secretory pathway, their prediction represents an integral part of subcellular-localization inference. However, experimental identification of SPs is labor-intensive and time-consuming, often requiring specialized reagents and being influenced by cell-type or condition-specific variability [5,6]. These limitations motivate the development of *in silico* prediction methods, which offer scalable, rapid, and reproducible alternatives for SP prediction [7]. The inference of subcellular localization via *in silico* methods plays a crucial role in functional annotation of uncharacterized proteins, bridging the gap between sequence data and biological insight [8].

Over the years, a wide range of methods have been implemented to predict SPs [9]. Starting from weight-matrix models [10], feature-based classifiers such as Support Vector Machines (SVMs) [11,12] and Random Forests [13], neural networks [14], probabilistic and homology-based models, such as Hidden Markov Models [15] and culminating in recent

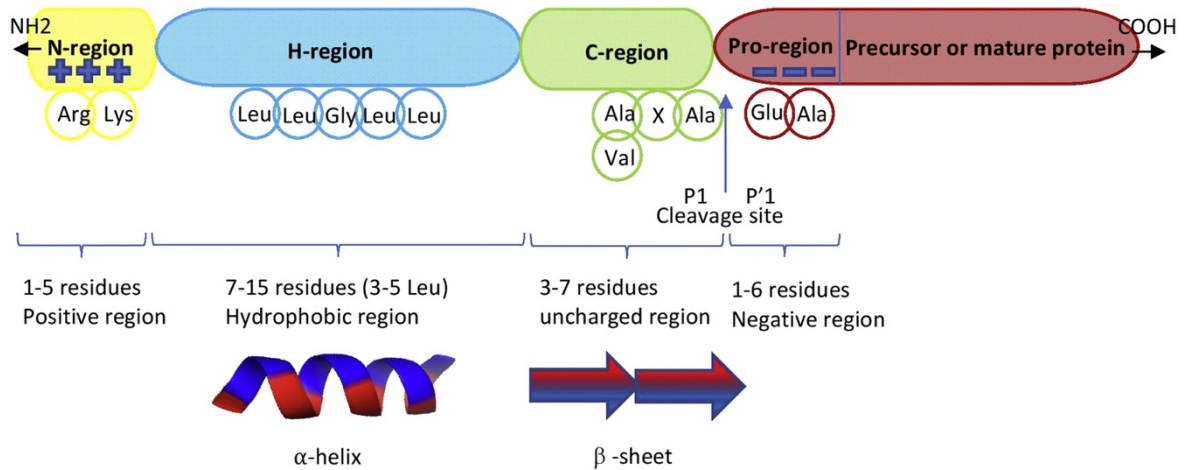


Figure 1. Schematic representation of a canonical SP. The SP contains an N-terminal region (1-5 residues), an H-region (7-15 residues), and a C-region (3-7 residues). Residues flanking the cleavage position are denoted P1 (upstream) and P'1 (downstream), and cleavage commonly occurs at motifs of the form A-X-A or V-X-A [3].

deep learning approaches [4,16]. Among these, the deep-learning approaches currently achieve the best overall performance in SP detection, as they learn complex, non-linear sequence patterns directly from raw sequence data and generalize better across species and SP types [4,16-18].

The aim of this project is to compare two computational approaches for SP detection, addressing a key component of protein function annotation and subcellular localization analysis. Specifically, the performance of von Heijne's (vH) positional scoring matrices and a machine-learning strategy based on SVMs were evaluated. By analyzing both methods on the same curated datasets, their ability to capture key properties of SPs and to generalize to unseen sequences was assessed. Overall, the results show that the vH method performs well on canonical motifs and the SVM achieves higher accuracy and MCC by integrating diverse sequence-derived features. Additionally, the benchmarking results confirm the cross-validation trends and highlighted characteristic false-positive and false-negative profiles for each method.

2 Materials and Methods

The project was implemented within the Python 3 environment, built on a modular structure that utilized key scientific computing libraries. Pandas and NumPy were essential for all aspects of data handling, including manipulation of the initial UniProtKB data, preparation of feature vectors, and efficient numerical operations. For model construction and evaluation, the Scikit-learn library was central, defining the machine learning pipeline, managing data scaling (StandardScaler), training the SVM classifier (specifically SVC), and performing rigorous performance assessment. Performance was evaluated using metrics derived from Scikit-learn, such as MCC and Accuracy, and was visually assessed by generating Precision-Recall (PR) curves. Furthermore, the project included a comparative baseline analysis using the vH algorithm, the implementation of which relied on NumPy for calculations. Finally, Matplotlib and Seaborn were employed across all stages for statistical visualization, enabling the creation of distribution plots and detailed performance charts.

2.1 Data Collection and Processing

The building of the dataset involved retrieving and curating high quality sequence data for both SP-positive and SP-negative examples from the UniProtKB database. Data retrieval was managed via the UniProt REST API employing a robust custom Python script to handle queries and data cleaning steps. The objective was to assemble a dataset of eukaryotic

proteins with experimentally confirmed SPs. To reduce noise, a set of filters was applied to the initial search. Sequences were restricted to Eukaryota, limited to reviewed proteins and required evidence at the protein level for existence. Furthermore, only sequences with an experimentally validated SP feature were included while short sequence fragments were excluded and a minimum sequence length of 40 residues was enforced. The initial UniProt query retrieved 2,949 entries (as of 22/09/2025). A critical step in data cleaning and processing was a custom filter implemented to address the SP length constraint. Since UniProtKB does not allow direct filtering of SPs by length, the retrieved JSON results were iterated over to ensure that only proteins possessing an SP of 14 residues or longer were retained. This final filtering step reduced the size of the positive dataset slightly resulting in 2,932 entries. The negative dataset was designed to represent non-secretory proteins with confirmed subcellular localization ensuring they were definitive negative examples. The same strict quality filters were applied (Eukaryota, reviewed, protein-level evidence, residues, non-fragments). The crucial exclusion filter was applied to remove any protein annotated with any evidence of SP presence. To further guarantee a non-secretory identity, proteins were required to have experimental evidence for localization in one of the following non-secretory compartments: cytosol, nucleus, mitochondrion, plastid, peroxisome, or cell membrane. The query for this dataset yielded 20,615 entries (as of 22/09/2025).

Subsequently, it was necessary to eliminate the occurrence of biases towards specific organisms and protein families, some of which comprise more sequences than others. All datasets were subjected to a standardized processing pipeline to reduce redundancy, ensure taxonomic diversity, and generate training and benchmarking subsets suitable for model development. Positive and negative sequences were clustered separately with MMseqs2 [19], using a minimum sequence identity of 30%, an alignment coverage of 40%, and connected component clustering mode, in order to obtain a higher generalization level. One representative sequence was retained from each cluster to avoid bias arising from closely related homologs. The representative sequences were then randomized and split into training (80%) and benchmarking (20%) sets, maintaining the same positive/negative ratio in both subsets. Training data was further split into five folds for cross-validation, with fold assignments recorded in the metadata file to ensure reproducibility and to prevent data leakage between training and validation steps. The sequences associated metadata was exported into a TSV file providing the following information for each sequence: UniProt

Accession Number, Organism, Lineage, Sequence Length, SP cleavage site position and evidence of a Transmembrane domain within the first 90 residues. In the TSV, the cleavage site column contains an integer representing the SP cleavage site position which equates to the SP length, set as 0 for negative sequences. Finally, the transmembrane domain evidence column reports “NIL” for positive sequences and a Boolean flag for negatives (False if the domain is not reported, True if it is). The final curated dataset statistics are summarized in **Table 1**.

A descriptive analysis was performed to assess the composition of the datasets and to verify that the training/benchmarking split did not introduce systematic bias. Both the training and benchmarking sets showed similar taxonomic distributions, with Metazoa as the most represented kingdom, followed by Fungi and Viridiplantae, and *Homo sapiens* as the dominant species in both. Protein-length distributions were comparable across training and benchmarking sets, indicating an unbiased split; although negative sequences showed a slightly broader length distribution, most proteins in both positive and negative datasets fell within the 200–500 residue range. Among positive sequences, SP length ranged from approximately 15 to 60 amino acids and peaked at 20–25 residues, consistent with canonical SPs, with similar distributions in both training and benchmarking datasets. Residue-composition analysis profiles further highlighted typical SP features: apolar residues, particularly alanine and leucine, were strongly overrepresented (with leucine showing approximately two-fold enrichment), whereas charged residues were depleted relative to the amino-acid composition reported for the UniProtKB/Swiss-Prot database (available through the ProtScale resource at ExPASy) [20]; these patterns were consistent across training and benchmarking sets. Sequence-logo analysis, performed using 13 positions upstream and 2 downstream relative to the cleavage site, and generated using the WebLogo tool [21], revealed the expected hydrophobic core preceding the C-region and the characteristic A-X-A motif, with no appreciable differences between the two datasets. For further insight of these analyses consult Supplementary Materials (Figures S1 to S5).

Table 1. Composition of the benchmarking set and cross-validation training folds.

Dataset split	Negative sequences	Positive sequences	Total sequences
Benchmarking	1,787	219	2,006
Fold 1	1,430	175	1,605
Fold 2	1,430	175	1,605
Fold 3	1,429	175	1,604
Fold 4	1,429	175	1,604
Fold 5	1,429	174	1,603
Total	8,934	1,093	10,027

2.2 von Heijne

To establish a baseline for SP prediction, the classical weight matrix method, originally developed by von Heijne (1986) was implemented using a Python script. This computational approach relies on Position-Specific Weight Matrices (PSWM) to capture the statistical distribution of amino acid residues found in the SP motif. The implementation utilized a dataset of 15-residue sequence motifs that were extracted from the positive dataset and anchored at the cleavage site (13 residues upstream and 2 downstream relative to the cleavage site). This ensured that the residue positions in the analysis corresponded to specific functional regions of the SP (N-, H-, and C-regions). To construct the Position-Specific Probability Matrix (PSPM), a one-hot encoding strategy was applied. Each amino acid

in the extracted motifs was converted into a binary vector (1 for presence, 0 for absence), allowing for the systematic counting of residue occurrences at each position j within the motif. To address the issue of sparse data, where certain residues might not appear in specific positions in the training set, pseudocounts were introduced to ensure non-zero probabilities for all potential residues. The probability $M_{k,j}$ was calculated using the formula:

$$M_{k,j} = \frac{1 + \sum_{i=1}^N I(s_{i,j} = k)}{N + 20} \quad (1)$$

where N is the total number of sequences, the value 1 is the added pseudocount, and 20 represents the standard amino acid correction. This probability matrix was subsequently converted into a log-odds weight matrix (PSWM) to normalize against the background distribution. The weight $W_{k,j}$ for residue k at position j is defined as:

$$W_{k,j} = \log_2 \left(\frac{M_{k,j}}{b_k} \right) \quad (2)$$

where the background probabilities b_k were derived from the standard SwissProt amino acid composition.

In this framework, positive values indicate that a residue is evolutionarily conserved or functionally favorable at a specific position, while negative values suggest selection against that residue. To predict the presence of a SP in a query sequence X , a sliding window approach with a fixed size of 15 residues was employed. Since SPs are exclusively located at the N-terminus, the scanning process was restricted to the first 90 residues of each protein sequence rather than the entire length. The score for each window within this N-terminal region was derived by summing the corresponding weights from the PSWM:

$$Score(X|W) = \sum_{i=1}^L W_{x_{i,i}} \quad (3)$$

The maximum score obtained across all windows in a sequence was taken as the final sequence score. This scoring mechanism was implemented using NumPy for efficient matrix operations, and the optimal classification threshold was determined by maximizing the F1 score on the validation dataset.

2.3 Support Vector Machines

SVMs are algorithms that take numerical feature vectors as input and solve classification tasks. These are trained to find the hyperplane that separates data points into classes in the n -dimensional feature space. The vectors defining the maximum hyperplane margin in the feature space, are known as support vectors. Hard Margin SVMs perform a perfect separation of data points, which does not allow misclassification. To introduce flexibility for situations where data are not perfectly separable, Soft Margin SVMs are utilized, allowing points to lie within the margin and the model to tolerate outliers and misclassification.

The classification task is a pivotal one in the bioinformatics field and non-linearly separable classification problems are often faced (e.g. determining whether a protein is a SP-carrying protein or not). To solve such problems, the original feature space can be mapped in a higher-dimensional feature space where training points are assumed to be linearly separable. Feature-transformation functions tend to be complex, therefore applying them to the entirety of the data points would increase the complexity of the computation. Kernel functions are used to directly evaluate the inner products of the extended feature space, without the need of defining it, since SVMs only rely on inner products between vectors.

In this framework, a hyperplane in the feature space (4) and the constrained optimization problem (5) can be expressed mathematically as:

$$\langle \bar{w}, \bar{x} \rangle + b = 0 \quad (4)$$

where w is the weight vector orthogonal to the hyperplane and b is the bias term, representing the offset of the hyperplane with respect to the origin of the feature space.

$$\min \left(\frac{1}{2} \|w\|^2 + C \sum_i \xi_i \right) \quad (5)$$

$$\text{subjected to } y_i(w^T x_i + b) \geq 1 - \xi_i \text{ with } \xi_i \geq 0 \quad \forall i$$

Maximizing the margin is equivalent to minimizing this quadratic relation (5), where x_i is the i -th data point and y_i is the relative class label. The constraint states that a data point is correctly classified with at least a margin of $1 - \xi_i$, where ξ_i is the slack variable (distance of the point from the hyperplane in case of misclassification) and C is the regularization parameter, constituting a trade-off between the margin maximization and the error minimization.

The optimal solution to the quadratic optimization problem having linear inequality constraints can be found employing the Lagrange multiplier method and deriving the Dual Lagrangian (6):

$$\mathcal{L}(\alpha_i) = \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j) \quad (6)$$

$$\text{subjected to } 0 \leq \alpha_i \leq C \quad \forall i \text{ and } \sum_i \alpha_i y_i = 0$$

where the first term α_i stands for the Lagrange multipliers; the second term represents the optimization problem after the complete derivation of the original expression, where α_i, α_j are the Lagrangian multipliers and y_i, y_j the class labels of the i -th and the j -th data points, while $K(x_i, x_j)$ is the Kernel for i -th and the j -th data points. In the case of linearly separable problems, the linear Kernel is used. On the contrary, for non-linearly separable problems, other Kernel functions are applied (e.g. the Radial Basis Function – RBF – kernel).

For the purposes of this work, protein sequences were encoded into 39-dimensional feature vectors via a tailored python script. Particularly, the baseline features ($n = 20$) represent the aminoacidic composition with respect to the 40-residues long N-terminal fragment of each sequence. These features should reflect the different amino acid proportions, such as the higher proportion of hydrophobic residues (e.g. leucine and alanine) characterizing the H- and C-regions of SP-positive examples.

Other features, attributing physicochemical characteristics found to be discriminative between secreted and non-secreted proteins, were extracted using a sliding-window approach to account for their context-dependent nature. A feature set addressing hydrophobicity ($n = 3$) and aliphaticity ($n = 3$) was extracted with a window size of 6, where hydrophobicity features were based on the Kyte–Doolittle hydrophobicity scale [22]. An additional set representing secondary structure propensity ($n = 3$ for α -helix and $n = 3$ for β -sheet) was extracted with a window size of 9, based on the α -helix and β -sheet amino acid scales [23,24]. A further set accounting for residue charges ($n = 4$) was extracted using a window size of 3. Finally, features describing transmembrane helix propensity ($n = 3$), based on the transmembrane tendency amino acid scale [25], were extracted with a window size of 7.

To classify proteins as SP-carrying ($y = 1$) or not ($y = 0$), a nonlinear SVM classifier was built through a five cross-validation procedure. The SVM was trained on the training data folds, yielding a grid search over several combinations of C and γ hyperparameters using the validation folds. The final model was selected based on the MCC performance assessed classifying the testing fold. During the training, a Random Forest (RF) classifier adjusted for class imbalance was applied to assess Gini feature importances. The SVM's MCC was evaluated for progressively

increasing feature sets to identify and retain the optimal discriminative subset of features.

2.4 Model Selection

2.4.1 von Heijne

For the vH-based classifier, model selection aimed to choose the optimal threshold to classify sequences as positive or negative based on the PSWM score. For each cross-validation round, a PSWM was first derived from the 15-residue motifs centered on the cleavage site of positive training sequences, using SwissProt amino acid frequencies as background. Each 90-residue N-terminal fragment in the corresponding validation fold was then scanned with a sliding 15-residue window, and the maximum PSWM score was retained as the sequence-level score. Precision–recall curves were computed from these scores, and the decision threshold was chosen as the value that maximized the F1-score, thus balancing precision and recall under class imbalance. The selected threshold was subsequently applied to the testing data fold to classify sequences as SP positive or negative and to assess performance metrics. This procedure was repeated across the five cross-validation rounds, and the resulting thresholds and metrics were averaged to obtain a robust estimate of the operating point for the vH model.

2.4.2 SVM

Once all sequences were encoded into scaled-feature vectors, within a five-fold cross-validation framework, an exhaustive grid search was performed over several combinations of diverse values of the C ($= 0.1, 1, 10, 100$) and γ ($=$ “scale”, $0.01, 0.1, 1.0$) hyperparameters. These hyperparameters, if efficiently selected, minimize the risk of overfitting. In particular, the regularization parameter C constitutes a tradeoff between the selection of the best separating hyperplane and the minimization of the classification errors. Smaller values of C are attributed to a more generalized model, characterized by a higher tolerance to misclassification, while higher values of C prioritize a stricter model, potentially leading to overfitting. Besides, lower values of γ make the model less sensitive to small data fluctuations, while higher values enhance sensitivity to local variation and increase the risk of overfitting.

For each C and γ combination, a model was trained on the training data folds ($n = 3$) and then used to predict the validation data fold to assess the MCC performance. The parameter set yielding the highest MCC on the testing fold was stored.

To determine the Gini feature importances, a class weighted RF classifier was trained with the following parameters: *number of estimators* = 400, *random state* = 42, *number of jobs* = 1, *class weights* = {0:9, 1:1}.

The features were ranked with respect to the importances in a decreasing order. Based on the ranking, incremental size (1 to 30) best-features subsets were built. For each subset, a SVM was trained on the training folds using the best parameter combination from the first grid search, and the MCC was evaluated on the validation fold to select the best-performing feature subset. Finally, a second grid-search was performed: the model was trained on the selected features and the final hyperparameters combination was selected based on the validation-set MCC.

2.5 Performance Scoring

For the binary classification task – prediction of SPs – performance evaluation was conducted using a defined set of standard metrics. These quantitative measures are essential for assessing model efficacy, comparing different algorithms and identifying potential biases. All metrics are derived from the statistical outcomes summarized in the Confusion Matrix which presents the observed class against the predicted class. This (2×2)

matrix yields four fundamental outcomes, True Positives (TP) where secreted proteins are correctly classified, True Negatives (TN) where non-secreted proteins are correctly classified, False Positives (FP) where non-secreted proteins are incorrectly classified as positive (Type I error) and False Negatives (FN) where secreted proteins are incorrectly classified as negative (Type II error).

The derived metrics provide a comprehensive measure of the classifier's performance with a particular emphasis on those that are robust against class imbalance.

The Matthews Correlation Coefficient (MCC) is considered the most reliable and balanced metric for binary classification tasks, as it produces a high score only if the classifier performs well across all four categories of the confusion matrix (TP, TN, FP, FN). Its inherent resilience to skewed data makes it the preferred measure for biological datasets. MCC values range from -1 (perfect inverse correlation) to $+1$ (perfect prediction). It is formally defined as:

$$MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (7)$$

The overall predictive power is first quantified by Accuracy (ACC), which represents the proportion of total correct predictions:

$$Accuracy = \frac{(TP + TN)}{(TP + TN + FP + FN)} \quad (8)$$

Model performance in relation to the positive class is further decomposed using Precision and Recall. Precision (Positive Predictive Value - PPV) quantifies the reliability of a positive prediction measuring the ratio of correctly predicted positive observations to the total predicted positive observations:

$$PPV = \frac{TP}{(TP + FP)} \quad (9)$$

Recall (Sensitivity or True Positive Rate - TPR) measures the model's completeness quantifying the ratio of correctly predicted positive observations to all observations in the actual positive class:

$$TPR = \frac{TP}{(TP + FN)} \quad (10)$$

These two metrics are combined into the F1 Score, which is their harmonic mean. It provides a single balanced measure of performance:

$$F1\ score = \frac{2 \times PPV \times TPR}{(PPV + TPR)} \quad (11)$$

3 Results

3.1 Cross-Validation

For each of the five runs of the five-fold cross-validation procedure performed to select the optimal classification threshold for the vH method, testing sequences were scored using the PSWM obtained from the training sequences and classified according to the optimal threshold selected from the corresponding validation fold (averaged PSWM is shown in **Figure S6**). For each run of the cross-validation, a PR-curve was obtained and shown in a final plot (**Figure S7**). The averaged optimal threshold obtained through the cross-validation procedure was $th = 8.809 \pm 0.597$, and the optimal threshold values are summarized in **Table 2**. The average of TP ($n = 638$), FP ($n = 309$), TN ($n = 6,838$), FN ($n = 236$) predicted in the cross-validation pipeline for the vH method is summarized in a confusion matrix (**Figure S8**).

For each of the five runs of the five-fold cross-validation procedure performed to select the optimal hyperparameters combination and feature subset for the SVM model, testing sequences were classified using the final model, trained on selected features and which hyperparameters were chosen through a grid search on the validation set over all hyperparameters combinations. The combinations selected for each fold are summarized in

Table 3. The average of TP ($n = 699$), FP ($n = 129$), TN ($n = 7,018$), FN ($n = 175$) predicted in the cross-validation pipeline for the vH method is summarized in a confusion matrix (**Figure S9**).

Table 2. Per-fold optimal threshold values obtained in the five-fold cross-validation procedure for the training of the vH method.

Fold	Optimal Threshold
1	8.989
2	8.486
3	8.068
4	8.657
5	9.844
Average	8.809 ± 0.597

Table 3. Per-fold number of selected features and optimal hyperparameter combination relative to the five-fold cross-validation procedure to select the best SVM model using selected features.

Fold	Selected Features	C	γ
1	26	1	0.01
2	25	10	0.01
3	29	10	0.01
4	25	10	scale
5	28	100	0.01

In **Table 4**, the averaged performance metrics of the vH and SVM cross-validation results are outlined. The SVM outperformed the vH overall, showing better prediction and generalization potential.

Table 4. Averaged performance metrics for the two methods across the five-fold cross-validation framework. Each metric averaged value is indicated accompanied by the relative standard deviation.

Metric	vH	SVM
<i>Accuracy</i>	0.932 ± 0.009	0.962 ± 0.003
<i>Precision</i>	0.683 ± 0.065	0.849 ± 0.034
<i>Recall</i>	0.730 ± 0.56	0.796 ± 0.041
<i>F1 score</i>	0.701 ± 0.023	0.821 ± 0.016
<i>MCC</i>	0.666 ± 0.026	0.801 ± 0.018

3.2 Benchmarking

Aiming to evaluate the performance of the final models, these were trained on the whole training dataset and then tested on the hold-out data. The vH performance evaluation was carried out building the PSWM matrix (**Figure S10**) on the entirety of the positive sequences belonging to the training dataset. Next, the benchmark dataset sequences were scored, and classified with respect to the optimal threshold selected via the five-fold cross-validation procedure. Overall, the performance showed to be consistent with the expected results.

The performance of the SVM was assessed by running the complete feature selection pipeline involving the employment of the RF on 4 training set folds and using the remaining one as the validation fold to select the best features subset and hyperparameter combination. A total of 29 features were selected (top 20 listed in **Figure S11**), and the best

hyperparameters combination was $C = 10$ and $\gamma = 0.01$, which happens to be the combination of the most frequently selected parameters in the cross-validation framework. Finally, the resulting model was tested on the hold-out set. As shown in **Table 5**, the SVM was confirmed to be outperforming the vH, yielding high MCC (which performance is detailed in **Figure S12**) and F1 score values, which are particularly meaningful under strong class imbalance and indicate that the model achieved reliable classification performance across both classes.

Table 5. Performance metrics for the two methods evaluated in the benchmarking framework.

Model	Accuracy	Precision	Recall	F1 score	MCC
vH	0.931	0.661	0.758	0.706	0.670
SVM	0.968	0.863	0.836	0.849	0.831

As shown in Table 6, where the TP, FP, TN, FN numbers for both methods are summarized, the SVM was able to correctly predict more sequences of the hold-out dataset overall. Additionally, the relative confusion matrices are depicted in **Figures S13** (vH) and **Figure S14** (SVM).

Table 6. TP, FP, TN, FN counts for the two methods in the benchmarking framework.

Model	TP	FP	TN	FN
vH	166	85	1,702	53
SVM	179	35	1,752	40

3.3 FP and FN Analysis

The prediction results were thoroughly analyzed for both models to assess the causes of the misclassifications. FP correspond to non-SP carrying sequences predicted as SP-carrying sequences. For both the vH and the SVM results, the overall False Positive Rate (FPR) was computed. Subsequently, it was evaluated with respect to negative sequences that were annotated as ones that exhibit a transmembrane domain within the first 90 residues. As shown in **Table 7**, in both cases, the FPR with respect to the presence of a transmembrane domain in the N-terminal of the sequence resulted to be higher compared to the overall FPR, highlighting that the physicochemical characteristics of these domains cause the misclassification, since they resemble the ones of SPs. Particularly, for the vH model the transmembrane domain-related FPR is almost five times the overall FPR, whereas for the SVM it is close to 3.5 times the overall FPR. On top, these results indicate once more that the SVM outperforms vH, showing an overall FPR less than half and a transmembrane domain-related FPR about a third of the vH one.

For the vH method, the sequence logo was computed for all 15-residue fragments of the FNs in the benchmark dataset. It is compared to the expected one in **Figure 2**, where it's evident that FNs may be due to a different composition of the cleavage-site context than expected.

For the SVM model, the amino acid compositions of positive examples from training sets up to 90 residues of benchmarking-FNs were compared to the benchmarking-TP one, as shown in **Figure 3**. As expected, these two subsets of hold-out data show differences in their compositions that are relevant in the context of SP prediction. Non-polar residues like alanine and leucine are less represented in FNs compared to TPs, determining misclassification.

Another source of misclassification was identified in the SP length distribution differences among benchmarking FNs and TPs, as detailed in

Figure 4. Particularly, FNs presented a broader cleavage site length distribution, indicating the presence of longer SP in this positive data subset, which resulted in misclassification.

Table 7. Summary of overall FPR and transmembrane domain-related FPR for the vH and SVM models.

Model	Overall FPR	TM domain FPR
vH	0.048	0.213
SVM	0.020	0.071

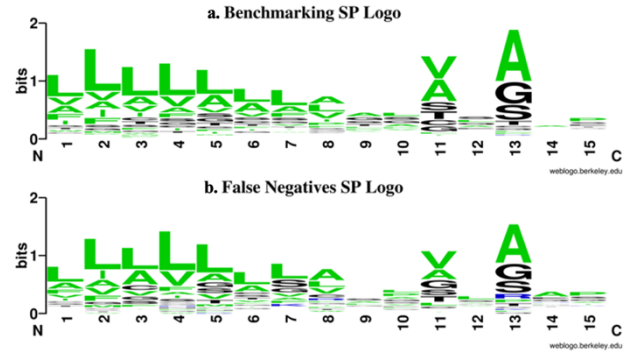


Figure 2. Sequence logos for the 15-residue motifs of: (a) Benchmark dataset, and (b) False Negatives resulting from the benchmarking procedure.

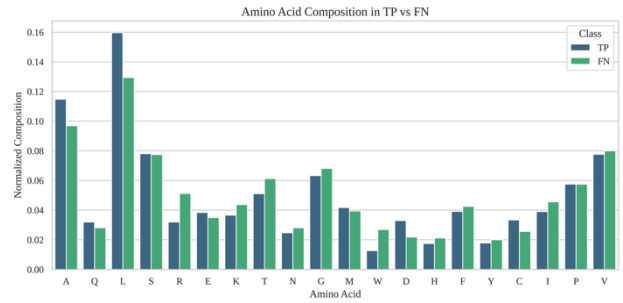


Figure 3. Amino acid composition of true-positive and false-negative sequences in the benchmarking dataset.

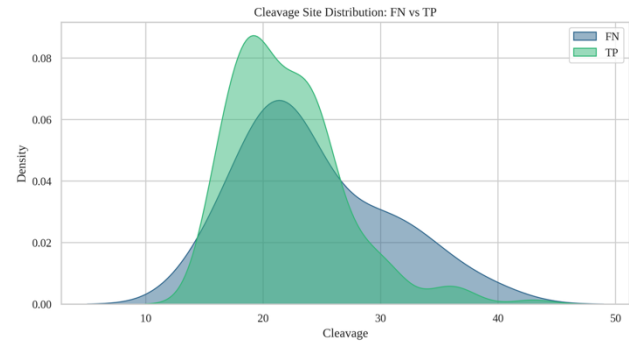


Figure 4. Distribution of SP lengths for true-positive and false-negative sequences in the benchmarking dataset.

Finally, as shown in **Figure S15** of supplementary materials, benchmarking-FNs present different feature trends compared to benchmarking-TPs: they show less hydrophobic patterns, a smaller α -helix and β -sheet tendency, and they also differ with respect to transmembrane domain tendency.

4 Conclusion

In this study, a systematic comparison of two complementary computational strategies for predicting SP in eukaryotic proteins was conducted, the classical vH positional scoring matrix approach and a machine-learning method based on SVM. By assembling a rigorously curated dataset of experimentally validated secretory and non-secretory proteins and by enforcing strict quality filters and controlled train test separation, we ensured that the benchmarking results reflected each model's true generalization capacity rather than dataset artefacts.

The analysis revealed that while the vH method reliably captures conserved sequence patterns around the cleavage site and performs well on canonical SPs, its predictive power is limited by its reliance on local positional statistics and its reduced sensitivity to broader physicochemical trends. In contrast, the SVM classifier integrating compositional, physicochemical, structural-propensity and context dependent features demonstrated substantially improved performance across cross-validation and benchmark tests. The gains in MCC, F1-score and precision highlight the advantage of incorporating diverse sequence derived descriptors when discriminating SPs from structurally similar motifs such as N-terminal transmembrane segments. Feature importance analysis further confirmed that hydrophobicity patterns, secondary-structure propensities and transmembrane tendency scores provide strong discriminative contributions and justify their inclusion in modern SP prediction pipelines.

Error analysis of both classifiers underscored biologically meaningful sources of misclassification. False positives were disproportionately enriched for proteins carrying transmembrane helices near the N-terminus reflecting the inherent physicochemical similarity between these helices and SP hydrophobic cores. False negatives, on the other hand, were frequently associated with atypical SP compositions or broader cleavage-site distributions which deviate from canonical SP motifs. These observations not only validate the models' behavior but also highlight subclasses of biologically challenging SP that require more expressive models to capture their diversity.

Taken together, these findings demonstrate that classical motif-based approaches retain value as transparent and computationally lightweight baselines but that machine-learning models leveraging broader feature spaces offer superior accuracy and robustness. As deep-learning methods increasingly dominate the field, this comparative framework provides a useful reference for how incremental increases in model complexity translate into predictive gains. The results presented here reinforce the importance of curated datasets, rigorous evaluation strategies and detailed error profiling in the development of reliable tools for protein-localization prediction. Ultimately, improving the accuracy of SP detection contributes directly to more precise protein annotation, supports large-scale functional genomics efforts and enhances downstream applications ranging from recombinant protein engineering to disease-related variant interpretation.

Acknowledgements

We wish to acknowledge Professors Castrense Savojardo and Matteo Manfredi for their expert advice, insightful feedback, and invaluable guidance throughout the completion of this work.

Funding

This work has been supported by the Alma Mater Studiorum University in Bologna.

Conflict of Interest: none declared.

References

- Alberts B., et al. (2022) *Molecular Biology of the Cell*, 7th ed. W. W. Norton & Company.
- Brown D., Breton S. (2000) Sorting proteins to their target membranes. *Kidney Int.*, **57**, 816–824.
- Owji H., et al. (2018) A comprehensive review of signal peptides: Structure, roles and applications. *Eur. J. Cell Biol.*, **97**, 422–441.
- Teufel F., et al. (2022) SignalP 6.0 predicts all five types of signal peptides using protein language models. *Nat. Biotechnol.*, **40**, 1023–1025.
- Zhang S., et al. (2025) Signal peptides: From molecular mechanisms to applications in protein and vaccine engineering. *Biomolecules*, **15**, 897.
- Burdukiewicz M., et al. (2018) Prediction of signal peptides in proteins from *Malaria* parasites. *Int. J. Mol. Sci.*, **19**, 3709.
- Ghovvati S., et al. (2018) *In silico* analysis of signal peptides for secretory production of interferon- β 1b. *Acta Biochim. Pol.*, **65**, 521–534.
- Chen J., et al. (2023) *In silico* protein function prediction: the rise of machine-learning-based approaches. *Med. Rev.*, **3**, 487–510.
- Nielsen H., et al. (2019) A brief history of protein sorting prediction. *Protein J.*, **38**, 200–216.
- von Heijne G. (1986) A new method for predicting signal sequence cleavage sites. *Nucleic Acids Res.*, **14**, 4683–4690.
- Vert J.P. (2002) Support Vector Machine prediction of signal peptide cleavage sites. *Pac. Symp. Biocomput.*, **7**, 649–660.
- Cai Y.D., Lin S.L., Chou K.C. (2003) Support vector machines for prediction of signal sequences and their cleavage sites. *Peptides*, **24**, 159–161.
- Kandaswamy K.K., et al. (2010) SPRED: A machine-learning approach for secretory protein identification. *Biochem. Biophys. Res. Commun.*, **391**, 1306–1311.
- Petersen T.N., et al. (2011) SignalP 4.0: discriminating signal peptides from transmembrane regions. *Nat. Methods*, **8**, 785–786.
- Barash S., et al. (2002) Human secretory signal peptide description by hidden Markov models. *Biochem. Biophys. Res. Commun.*, **294**, 835–842.
- Savojardo C., et al. (2018) DeepSig: Deep learning improves signal peptide detection. *Bioinformatics*, **34**, 1690–1696.
- Almagro Armenteros J.J., et al. (2019) SignalP 5.0 improves signal peptide predictions using deep neural networks. *Nat. Biotechnol.*, **37**, 420–423.
- Dumitrescu A., et al. (2023) TSignal: A transformer model for signal-peptide prediction. *Bioinformatics*, **39**, i347–i356.
- Steinegger M., Söding J. (2017) MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nat. Biotechnol.*, **35**, 1026–1028. doi:10.1038/nbt.3988.
- Bairoch A. (2013) Amino-acid composition (%) in the UniProtKB/Swiss-Prot data bank. *UniProtKB/Swiss-Prot Release Notes*, Release 2013_04. Available at: ExPASy ProtScale (<https://web.expasy.org/protscale/pscale/A.A.Swiss-Prot.html>).
- Crooks G.E., Hon G., Chandonia J.-M., Brenner S.E. (2004) WebLogo: a sequence logo generator. *Genome Res.*, **14**, 1188–1190. doi:10.1101/gr.849004. Available at: <https://weblogo.berkeley.edu>
- Kyte J., Doolittle R.F. (1982) Amino-acid hydropathicity scale. *J. Mol. Biol.*, **157**, 105–132. Available at: ExPASy ProtScale (<https://web.expasy.org/protscale/pscale/Hphob.Doolittle.html>).
- Chou P.Y., Fasman G.D. (1978) Conformational parameters for α -helix formation. *Adv. Enzymol.*, **47**, 45–148. Available at: ExPASy ProtScale (<https://web.expasy.org/protscale/pscale/alpha-helixFasman.html>).
- Chou P.Y., Fasman G.D. (1978) Conformational parameters for β -sheet formation. *Adv. Enzymol.*, **47**, 45–148. Available at: ExPASy ProtScale (<https://web.expasy.org/protscale/pscale/beta-sheetFasman.html>).
- Zhao G., London E. (2006) Amino-acid transmembrane tendency scale. *Protein Sci.*, **15**, 1987–2001. Available at: ExPASy ProtScale (<https://web.expasy.org/protscale/pscale/Transmembranetendency.html>).